Bitácora S07 — Mexi Banco

Alumno: Diego Alexander Carrasco Quiñones

Fecha de entrega: `12/10/2026`

1. De una llamada a un método a una petición HTTP

Fecha: 12 de octubre de 2026

En esta práctica aprendí que, cuando un servicio necesita comunicarse con otro, ya no basta con llamar directamente a un método. Como los servicios están separados, deben comunicarse mediante peticiones HTTP.

Esto significa que el servicio al que se realiza la petición debe estar disponible, responder correctamente y hacerlo dentro del tiempo esperado. Si existe un problema de comunicación, la operación puede fallar aunque el servicio que la inició esté funcionando.

Durante las pruebas de Mexi Banco, al intentar crear la cuenta de Ana con un saldo inicial de `1000`, se obtuvo un error `503 Service Unavailable`.

Resultado obtenido:


```
{
  "type": "about:blank",
  "title": "Service Unavailable",
  "status": 503,
  "detail": "No se pudo contactar a otro servicio: Could not retrieve response status
code: java.net.SocketTimeoutException: Read timed out",
  "instance": "/cuentas"
}
```


El mensaje indica que no se pudo obtener a tiempo la respuesta de otro servicio. Esto demuestra que una operación que depende de varios microservicios puede fallar por un problema de comunicación.

2. Directorio de servicios con Eureka

Fecha: 12 de octubre de 2026

Utilicé Eureka para revisar qué servicios estaban registrados y cuántas instancias había disponibles. Al principio, el directorio todavía no mostraba los servicios necesarios. Después de iniciar la aplicación, comprobé que aparecían los siguientes:

Servicio	Instancias registradas
`CUENTA`	3
`GATEWAY`	1
`MOVIMIENTO`	1
`NOTIFICACION`	1
`SPEI`	1
`TRANSFERENCIA`	1

En total había ocho instancias registradas. `CUENTA` tenía tres, mientras que cada uno de los otros servicios tenía una.

También revisé la información de `CUENTA` en formato XML. Esto me permitió observar los datos que Eureka mantiene sobre las instancias registradas.

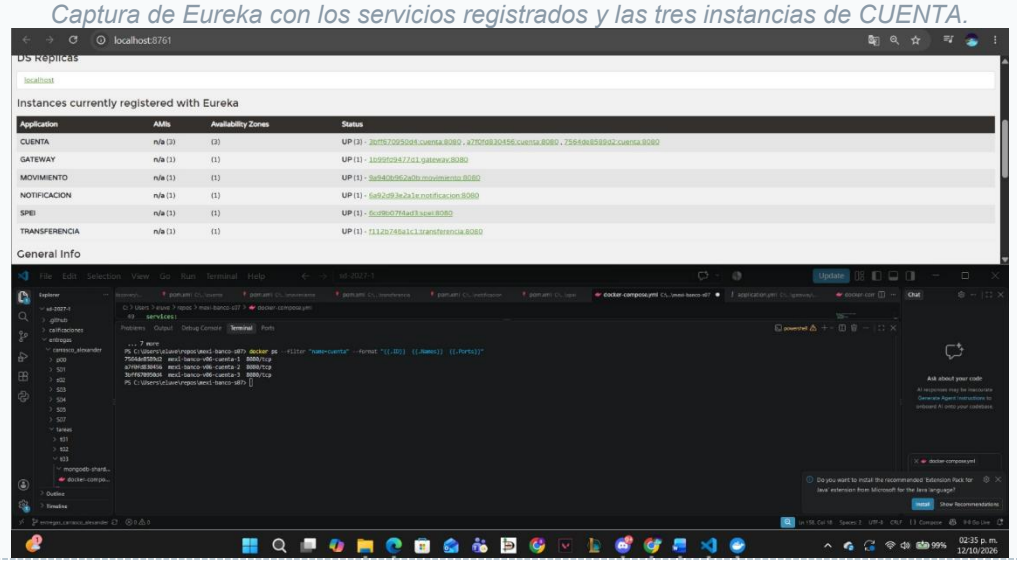
Entendí que Eureka funciona como un directorio que ayuda a los servicios a encontrarse sin depender únicamente de direcciones fijas. También permite identificar las instancias disponibles cuando un servicio tiene varias réplicas.

Evidencias que colocaré:

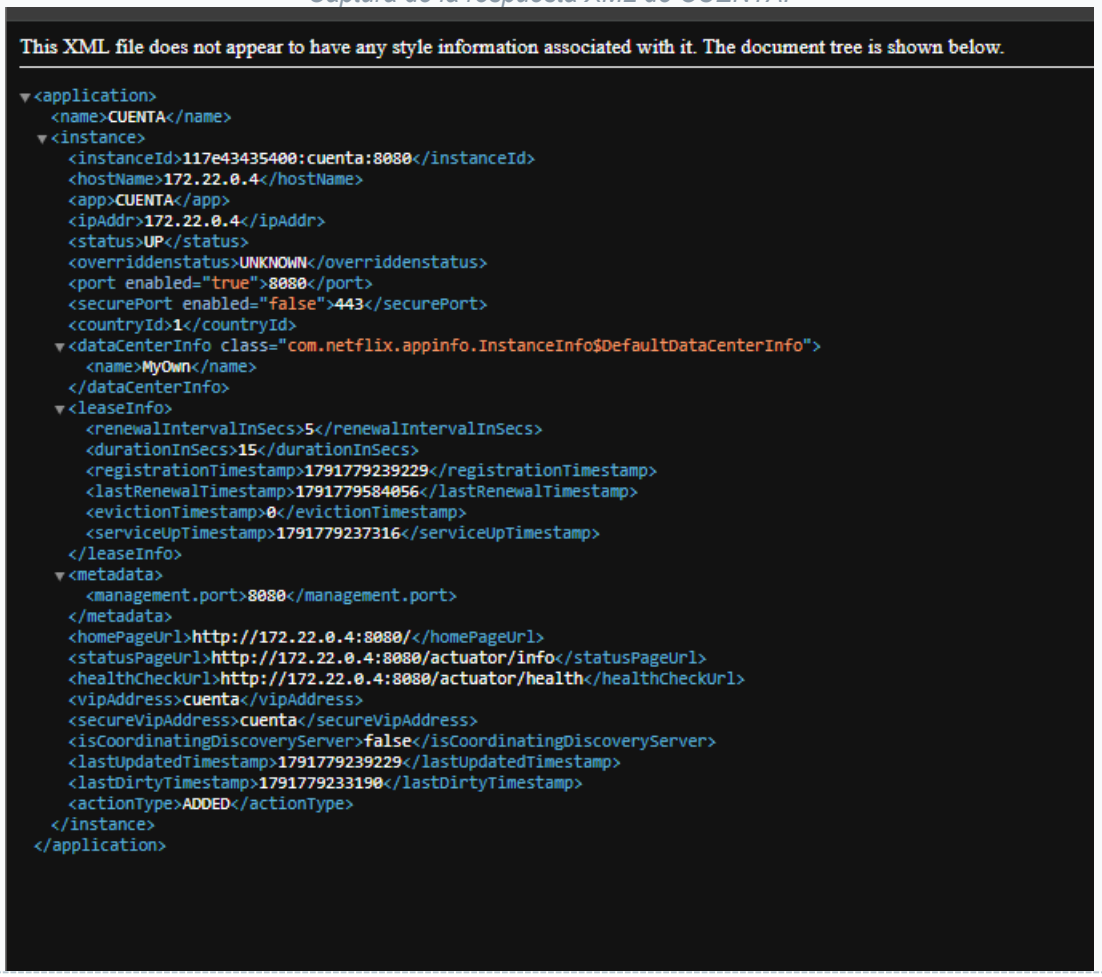
Captura de Eureka antes de que aparecieran los servicios.



Captura de Eureka con los servicios registrados y las tres instancias de CUENTA.



Captura de la respuesta XML de CUENTA.



Respuesta a la pregunta de reflexión: Eureka facilita el descubrimiento de servicios y permite conocer qué instancias están registradas. Esto ayuda a que los servicios puedan comunicarse y a que las peticiones se distribuyan entre las réplicas disponibles.

3. Pruebas HTTP mediante el Gateway

Fecha: 12 de octubre de 2026

Realicé diferentes peticiones mediante el Gateway para comprobar el funcionamiento de los servicios. Durante las pruebas observé respuestas HTTP `400`, `404`, `201` y `200`.

El código `400 Bad Request` apareció en una prueba realizada en el puerto `8082`. Este código indica que la solicitud fue rechazada por considerarse incorrecta.

También obtuve errores `404 Not Found`. Uno ocurrió al consultar la cuenta de Ana, que no existía en ese momento. Otro apareció al consultar la ruta `/api/transferencias/`, que no encontró un recurso para atender la petición.

Consulta de la cuenta de Beto

La cuenta de Beto se creó correctamente y después realicé una consulta mediante el Gateway. La salida de terminal que conservé fue:

```
PS C:\Users\eluve\repos\mexi-banco-s07> Invoke-RestMethod -Uri
"http://localhost:8080/api/cuentas/002180009184293002"

id clabe              titular saldo
-- -----              ------- -----
 3 002180009184293002 Beto    500.00
```

La respuesta mostró que la cuenta de Beto tenía el identificador `3` y un saldo de `500.00`. La consulta fue exitosa y correspondió a una respuesta HTTP `200 OK`.

Error al consultar la ruta de transferencias

Durante otra prueba se intentó acceder a:

```
http://localhost:8080/api/transferencias/
```

El resultado registrado fue:

```
404
No static resource transferencias
```

Este mensaje indica que la ruta solicitada no encontró un recurso disponible para atender la petición.

Intento de abonar a la cuenta de Ana

Después de crear la cuenta de Ana con saldo cero, intenté abonarle `1000` mediante el Gateway:

```
Invoke-RestMethod -Method Post -Uri
"http://localhost:8080/api/cuentas/002180009184293001/abonos" -ContentType
"application/json" -Body '{"monto":1000}'
```

La respuesta fue:

```
{"timestamp":"2026-10-
12T22:29:26.109+00:00","path":"/api/cuentas/002180009184293001/abonos","status":404,"error":"Not
Found","requestId":"2571d5ec-1468"}
```

La petición devolvió `404 Not Found`, por lo que el abono no se pudo realizar mediante esa ruta del Gateway.

En las pruebas de creación de Beto se obtuvo el código `201 Created`, mientras que su consulta devolvió `200 OK`. Los errores `404` fueron distintos: uno correspondía a una cuenta que no existía en ese momento y otro a una ruta que no encontró un recurso disponible.

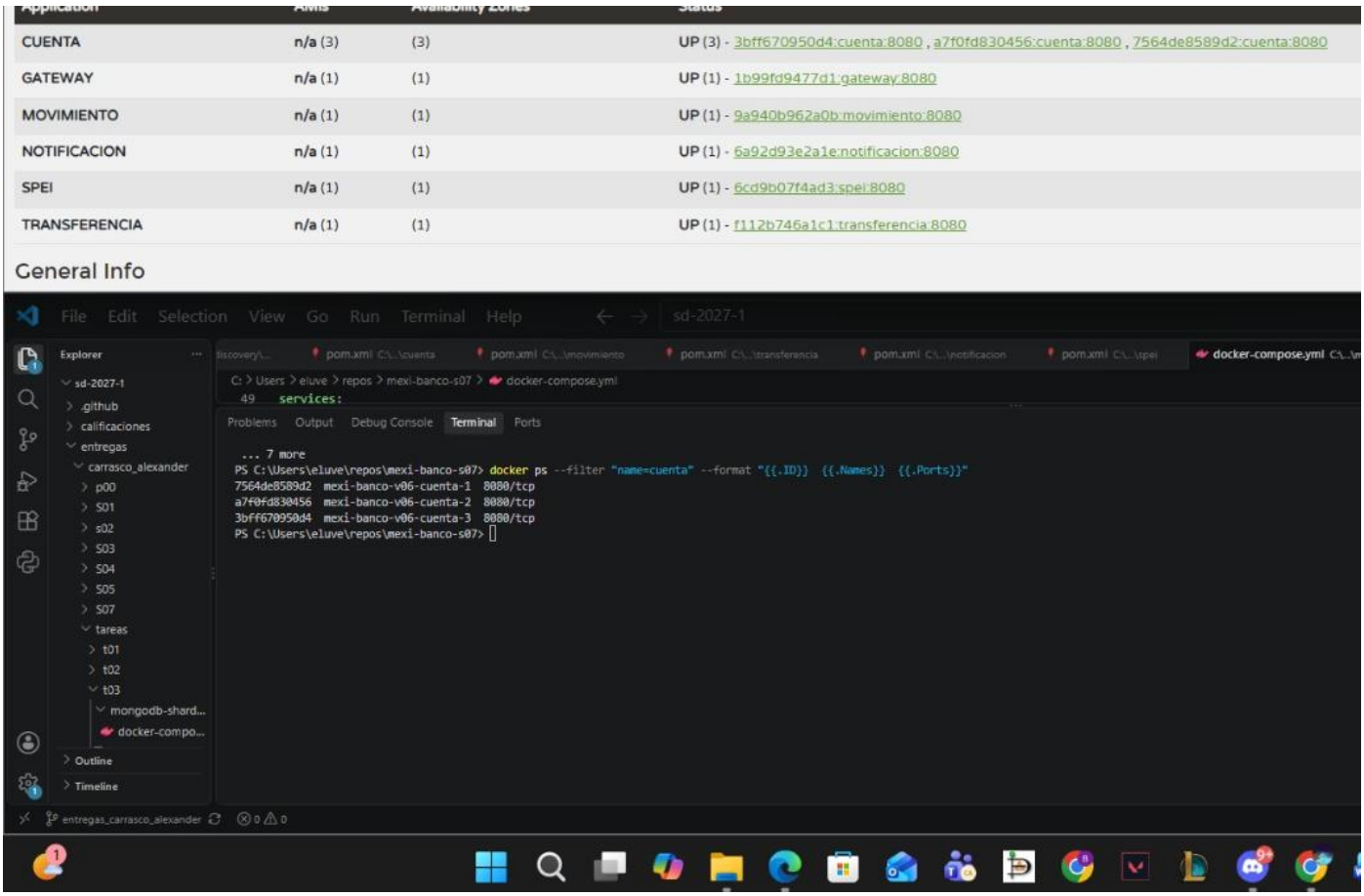
4. Réplicas y balanceo de carga

Fecha: 12 de octubre de 2026

Comprobé que `CUENTA` tenía tres instancias. Para identificar cuál respondía a cada petición, revisé la cabecera HTTP `X-Instancia`.

Los identificadores de instancia que quedaron registrados fueron:

```
7564de8589d2
a7f0fd830456
3bff670950d4
```



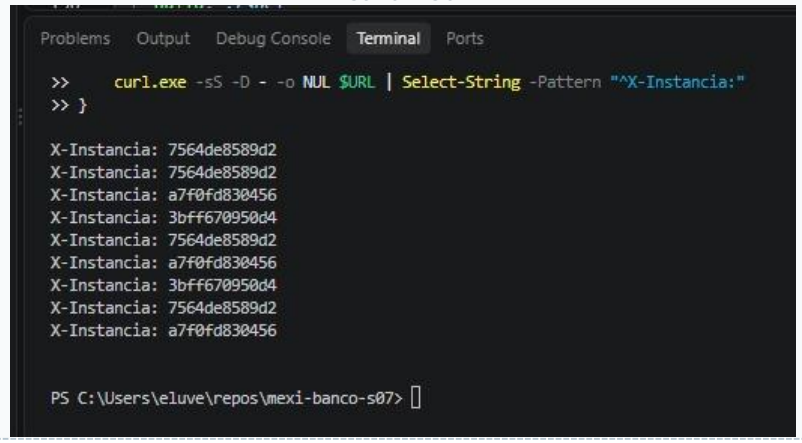
Durante las consultas observé que las respuestas podían provenir de diferentes instancias. Esto me ayudó a entender que el balanceo de carga distribuye las peticiones entre las réplicas disponibles, en lugar de depender siempre de una sola.

Antes de apagar una instancia, mi predicción era que las otras dos podrían seguir atendiendo peticiones, siempre que permanecieran disponibles y el sistema dejara de dirigir solicitudes a la instancia que hubiera fallado.

La información anterior identifica las tres instancias, pero no conserva la secuencia exacta de las nueve cabeceras `X-Instancia`.

Evidencia:

Round Robin



5. Apagado de una réplica y tolerancia a fallos

Fecha: 12 de octubre de 2026

Plazo de concesión configurado en Eureka: 15 segundos.

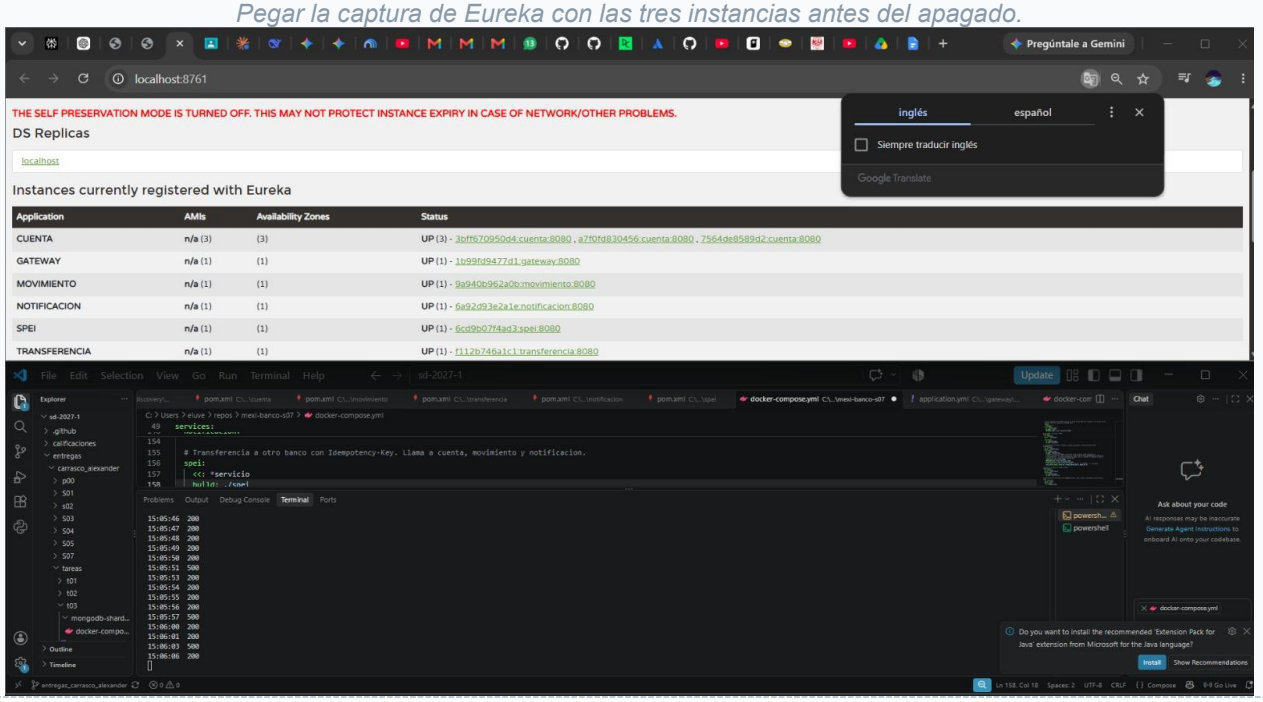
En este punto debía observar qué sucedía al apagar una instancia de `CUENTA`, cuánto tardaba en desaparecer de Eureka y cuántos errores se producían durante la prueba.

Mi predicción era que las otras réplicas podrían continuar atendiendo peticiones después de que el sistema detectara que la instancia apagada ya no estaba disponible. Sin embargo, durante el intervalo de detección podían producirse errores si alguna solicitud se dirigía a una instancia que había dejado de responder.

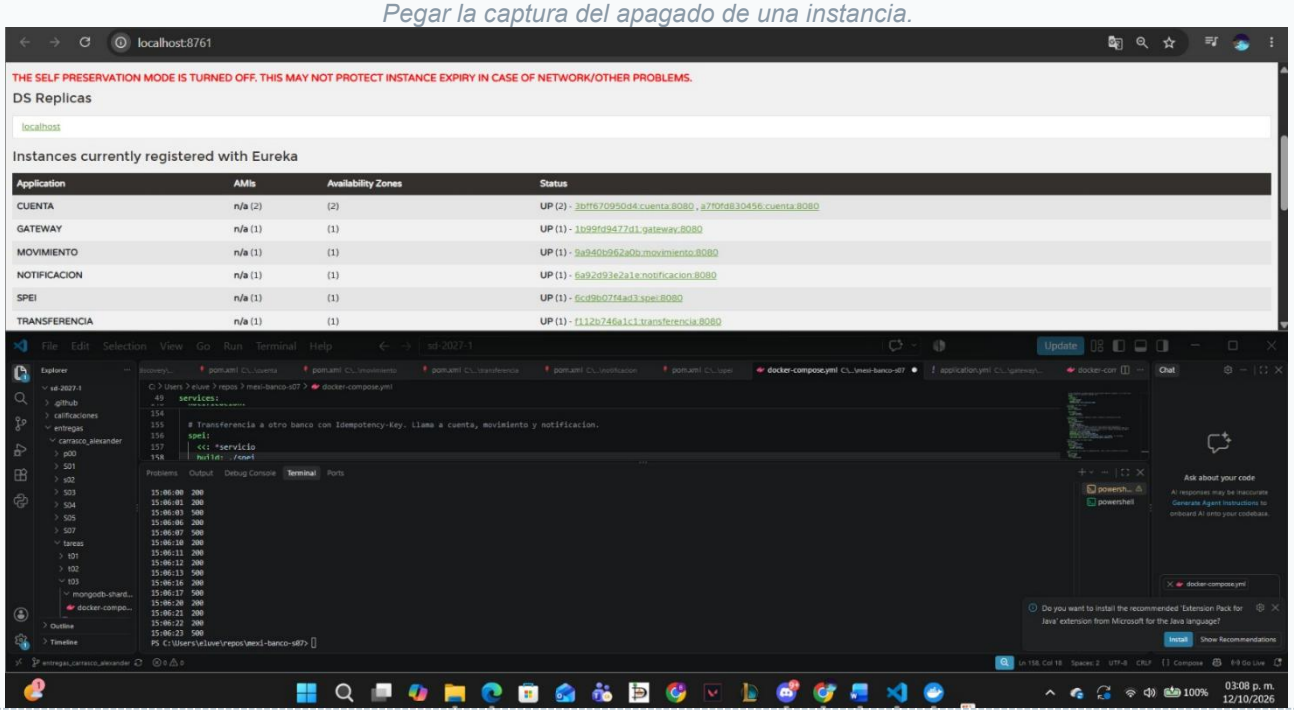
No se conservaron en el registro disponible las horas exactas del apagado y de la desaparición de Eureka, ni el número final de errores. Por eso, no asigno valores que no pueda comprobar.

Evidencias:

Pegar la captura de Eureka con las tres instancias antes del apagado.



Pegar la captura del apagado de una instancia.



Conclusión: tener varias réplicas ayuda a que un servicio pueda continuar funcionando cuando una instancia falla. Sin embargo, esto no significa que todos los errores desaparezcan inmediatamente, porque el sistema necesita detectar la falla y dejar de utilizar la instancia que ya no responde.

6. Ejecución de `demo-v06.sh`, verificación de datos y conclusión

Fecha: 12 de octubre de 2026

Para el reto final ejecuté el script `demo-v06.sh`. Como es un archivo de Bash, utilicé Git Bash desde PowerShell:

```
& "C:\Program Files\Git\bin\bash.exe" -lc 'cd /c/Users/eluve/repos/mexi-banco-s07 && bash ./demo-v06.sh'
```

Durante la ejecución intenté crear la cuenta de Ana con la CLABE `002180009184293001` y un saldo inicial de `1000`. La operación devolvió el error `503` que expliqué en el punto 1.

Después, intenté transferir `200` de Ana a Beto. La operación falló porque la cuenta de Ana no se encontraba en la base de datos de cuentas. También intenté realizar una operación SPEI de `50` utilizando la clave de idempotencia `spei-demo-91842930`. El primer intento falló porque no se encontró la cuenta y, al repetirlo con la misma clave, obtuve este mensaje:

```
409
"Ya hay un SPEI en vuelo con esta Idempotency-Key"
```

Esto indicó que el sistema reconocía que ya existía una operación en curso con esa clave.

Evidencia:

```
PS C:\Users\eluve\repos\mexi-banco-s07> & "C:\Program Files\Git\bin\bash.exe" -lc 'cd /c/Users/eluve/repos/mexi-banco-s07 && bash ./demo-v06.sh'

=== 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuenta ===
-> listo en 3 s.

Lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA 3 instancia(s)
GATEWAY 1 instancia(s)
MOVIMIENTO 1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI 1 instancia(s)
TRANSFERENCIA 1 instancia(s)

-> 404 de cuenta (no 503 del gateway) para una CLABE que no existe: la puerta ya sabe llegar


=== 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) ===

$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009184293001","titular":"Ana","saldoInicial":1000}'

{
  "type": "about:blank",
  "title": "Service Unavailable",
  "status": 503,
  "detail": "No se pudo contactar a otro servicio: Could not retrieve response status code: java.net.SocketTimeoutException: Read timed out",
  "instance": "/cuentas"
}
[HTTP 503]

-> el gateway quito /api y lo mando a lb://cuenta: a una de sus tres replicas


=== 2. Abrir la cuenta de Beto con 500 (misma puerta) ===

$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clabe":"002180009184293002","titular":"Beto","saldoInicial":500}'

{"id":3,"clabe":"002180009184293002","titular":"Beto","saldo":500}
[HTTP 201]
```

```
=== 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) ===

$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009184293001","claveDestino":"002180009184293002","monto":200}'

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009184293001",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]

-> transferencia llamo a http://cuenta (un nombre, sin puerto).
A que replicas fue cada llamada:
cuenta-3 GET /cuentas/002180009184293001 atendida por 3bff670950d4
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia


=== 4. Consultar esa transferencia ===

$ curl http://localhost:8080/api/transferencias/

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No static resource transferencias.",
  "instance": "/transferencias/"
}
[HTTP 404]
```

```
=== 5. Saldos: el dinero no se crea ni se pierde ===

$ curl http://localhost:8080/api/cuentas/002180009184293001

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009184293001",
  "instance": "/cuentas/002180009184293001"
}
[HTTP 404]

$ curl http://localhost:8080/api/cuentas/002180009184293002

{"id":3,"clabe":"002180009184293002","titular":"Beto","saldo":500.00}
[HTTP 200]

-> Ana + Beto 500.00 = 500.00 (antes: 1000 + 500 = 1500).
Esperado: 800 y 700


=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===

$ curl http://localhost:8080/api/movimientos?clabe=002180009184293001

[{"id":1,"clabeCuenta":"002180009184293001","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-12T22:09:03.879961Z","referencia":"saldo inicial"}]
[HTTP 200]

$ curl http://localhost:8080/api/notificaciones?clabe=002180009184293002

[]
[HTTP 200]
```

```
=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===

$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91842930' -d '{"claveOrigen":"002180009184293001","bancoDestino":"BANCO-
X","claveDestino":"012180000000000009","monto":50}'

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009184293001",
  "instance": "/spei",
  "servicio": "cuenta"
}
[HTTP 404]

$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91842930' -d '{"claveOrigen":"002180009184293001","bancoDestino":"BANCO-
X","claveDestino":"012180000000000009","monto":50}'

{
  "type": "about:blank",
  "title": "Conflict",
  "status": 409,
  "detail": "Ya hay un SPEI en vuelo con esta Idempotency-Key",
  "instance": "/spei"
}
[HTTP 409]

-> mismo id ( -> ): no se cobro dos veces.
Saldo de Ana: (esperado 750)


=== 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta ===

consulta 1 X-Instancia: 7564de8589d2 (mexi-banco-v06-cuenta-1)
consulta 2 X-Instancia: 3bff670950d4 (mexi-banco-v06-cuenta-3)
consulta 3 X-Instancia: a7f0fd830456 (mexi-banco-v06-cuenta-2)
consulta 4 X-Instancia: 7564de8589d2 (mexi-banco-v06-cuenta-1)
consulta 5 X-Instancia: 3bff670950d4 (mexi-banco-v06-cuenta-3)
consulta 6 X-Instancia: a7f0fd830456 (mexi-banco-v06-cuenta-2)
consulta 7 X-Instancia: 7564de8589d2 (mexi-banco-v06-cuenta-1)
consulta 8 X-Instancia: 3bff670950d4 (mexi-banco-v06-cuenta-3)
consulta 9 X-Instancia: a7f0fd830456 (mexi-banco-v06-cuenta-2)

-> 3 replicas distintas en 9 consultas; 0 veces se repitio la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce el 8080
```

```
=== 9. Los endpoints internos no tienen puerta ===

$ curl -X POST http://localhost:8080/api/cuentas/002180009184293001/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'

{"timestamp":"2026-10-12T22:09:46.656+00:00","path":"/api/cuentas/002180009184293001/abonos","status":404,"error":"Not Found","requestId":"1e043e38-1230"}
[HTTP 404]

-> 404: el gateway no tiene ruta para /cargos ni /abonos.
Adentro de la red siguen abiertos (seguridad: mas adelante)


=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===

$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009184293001","claveDestino":"002180009184293099","monto":10}'

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009184293001",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]

$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009184293001","claveDestino":"002180009184293002","monto":99999}'

{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009184293001",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]


=== 11. Los saldos no se movieron con los errores ===

-> Ana (esperado 750) + Beto 500.00 (esperado 700) = 500.00;
SPEI se llevo 50 a otro banco

Fin.

Para ver cuanto tarda el directorio en enterarse de una caida:
./demo-v06.sh --latido

PS C:\Users\eluve\repos\mexi-banco-s07>
```

Verificación de la base de datos de cuentas

Para comprobar qué cuentas habían quedado guardadas, ejecuté:

```
docker compose exec db psql -U mexibanco -d cuenta -c "SELECT id, clabe, titular, saldo
FROM cuenta ORDER BY id;"
```

El resultado fue:

```
 id |       clabe        | titular | saldo
----+--------------------+---------+--------
  1 | S071791782461      | S07     | 0.00
  3 | 002180009184293002 | Beto    | 500.00
(2 rows)
```

En ese momento, la tabla mostraba la cuenta de prueba S07 y la cuenta de Beto, pero no aparecía la cuenta de Ana.

Verificación de los movimientos

Después consulté los movimientos asociados a la CLABE de Ana:

```
docker compose exec db psql -U mexibanco -d movimiento -c "SELECT id, clabe_cuenta, tipo,
monto, saldo_resultante, referencia FROM movimiento WHERE clabe_cuenta =
'002180009184293001' ORDER BY id;"
```

El resultado fue:

```
 id |    clabe_cuenta    |   tipo   | monto   | saldo_resultante | referencia
----+--------------------+----------+---------+------------------+---------------
  1 | 002180009184293001 | DEPOSITO | 1000.00 | 1000.00          | saldo inicial
(1 row)
```

Con esto comprobé que el depósito inicial de `1000.00` sí se había guardado en la base de datos de movimientos, aunque la cuenta de Ana no se había creado correctamente. Esto dejó información parcial entre dos servicios.

Recuperación de la cuenta de Ana

Para volver a crear la cuenta sin registrar nuevamente el depósito inicial, utilicé un saldo de cero:

```
Invoke-RestMethod -Method Post -Uri "http://localhost:8080/api/cuentas" -ContentType
"application/json" -Body
'{"clabe":"002180009184293001","titular":"Ana","saldoInicial":0}'
```

La salida fue:

```
id clabe              titular saldo
-- -----              ------- -----
 4 002180009184293001 Ana         0
```

La cuenta se creó con identificador `4` y saldo cero.

Posteriormente intenté abonarle los `1000`, pero la petición devolvió `404 Not Found`, como se muestra en el punto 3. Por esta razón, no se pudo completar la recuperación mediante esa ruta.

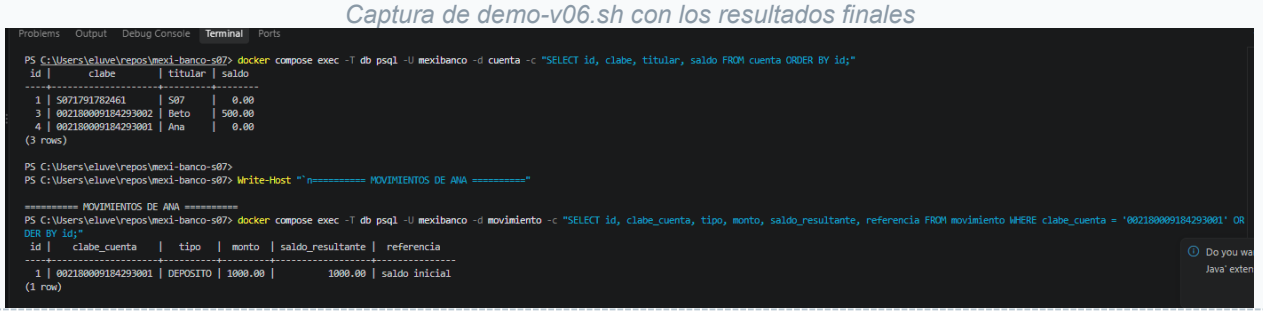
Resultado final del demo

Cuenta	CLABE	Resultado confirmado
Ana	`002180009184293001`	Se creó posteriormente con saldo `0`; el abono de 1000 devolvió 404.
Beto	`002180009184293002`	Consulta exitosa con saldo `500.00`.

El demo no terminó con la transferencia exitosa ni con el saldo esperado de Ana. Los resultados reales mostraron un error al crear la cuenta, un movimiento registrado sin la cuenta correspondiente y una operación de recuperación que no se pudo completar mediante el Gateway.

Evidencia:

Captura de demo-v06.sh con los resultados finales



Conclusión general

Con esta práctica aprendí que Eureka permite descubrir servicios, el Gateway centraliza el acceso a ellos y las réplicas ayudan a distribuir las peticiones. También comprobé que una operación que depende de varios servicios puede fallar parcialmente: en la prueba de Ana, el movimiento quedó registrado, pero la cuenta inicial no se guardó correctamente.

Por eso es importante revisar tanto las respuestas HTTP como los registros de las bases de datos. Tener varias instancias mejora la disponibilidad, pero no garantiza que todas las operaciones se completen correctamente ni que los datos de distintos servicios permanezcan consistentes.